

Hashing

Security Mechanism

What is Hashing?



Let's say you sign up on a new website using your email id and trusted password combination. How do you think it will be stored on their servers?



Let's say you sign up on a new website using your email id and trusted password combination. How do you think it will be stored on their servers?



Servers



Plaintext

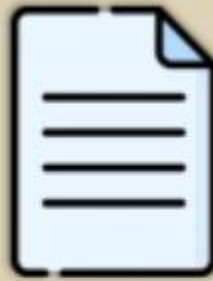


Encrypted



Hashed

Plaintext



- Very easy to gather information from.
- No security, can be reused with other websites.
- Website owner can also read personal data.

Encrypted



- More secure than plaintext.
- Decryption key is stored on servers, so it's still vulnerable.
- Once decryption key is hacked, all the encrypted passwords are broken.

Hashed



- Hashing will not reveal passwords in a data breach
- No encryption key needed
- Website owners can't read your data either

Hashed



Therefore, Hashing is the most ideal way
to store passwords.

- No encryption key needed
- Website owners can't read your data either

What Are We Learning Today?

1. What Is Hashing?
2. Real-World Implementation
3. Hash Functions
4. Hashing Guidelines
5. Ways to Strengthen Hashes (Salting and Peppering)



What Is Hashing?

Hashing is the process of scrambling a piece of information or data beyond recognition. They are designed to be **irreversible**. We pass the input through a hash function to calculate the **Hash Value** or **Digest**.

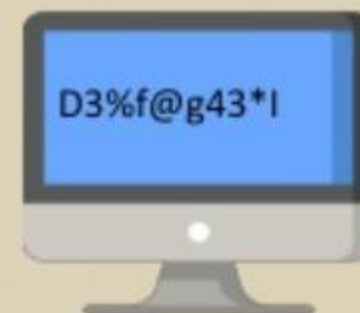


Original Data

+



Hash Function



Hash Value/Digest

Real-World Implementation

When a new user signs-up, the new password is passed through the **hash function** and the digest is stored on the **server**.



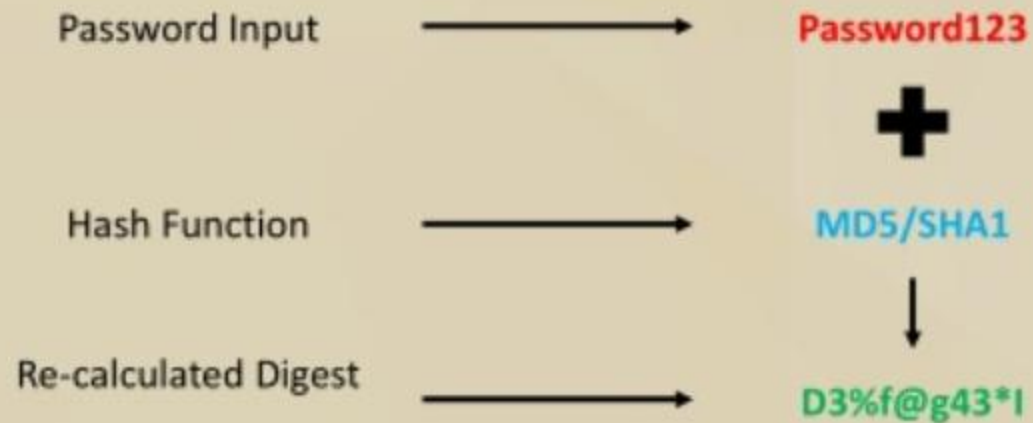
Hash stored on server



Real-World Implementation

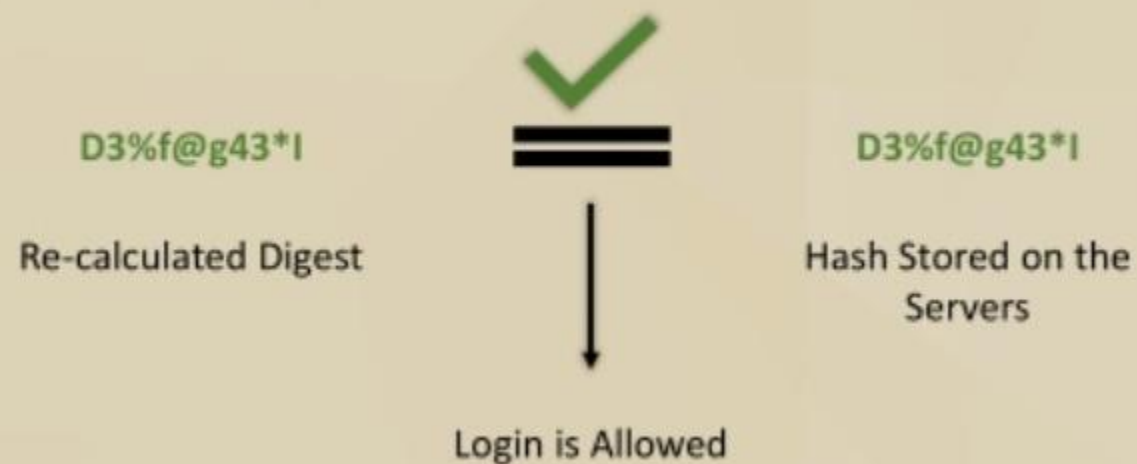
simplelearn

When the same user tries to log-in, the password they input is passed through the function again and the digest is compared to the one stored on the servers.



Real-World Implementation

If the re-calculated hash matches the hash stored on the servers during initial sign-up, the log-in is allowed.



Real-World Implementation

If the calculated digest is different from the one on the server, the login is denied from the website.

R2#h9Ln7q&

Re-calculated Digest



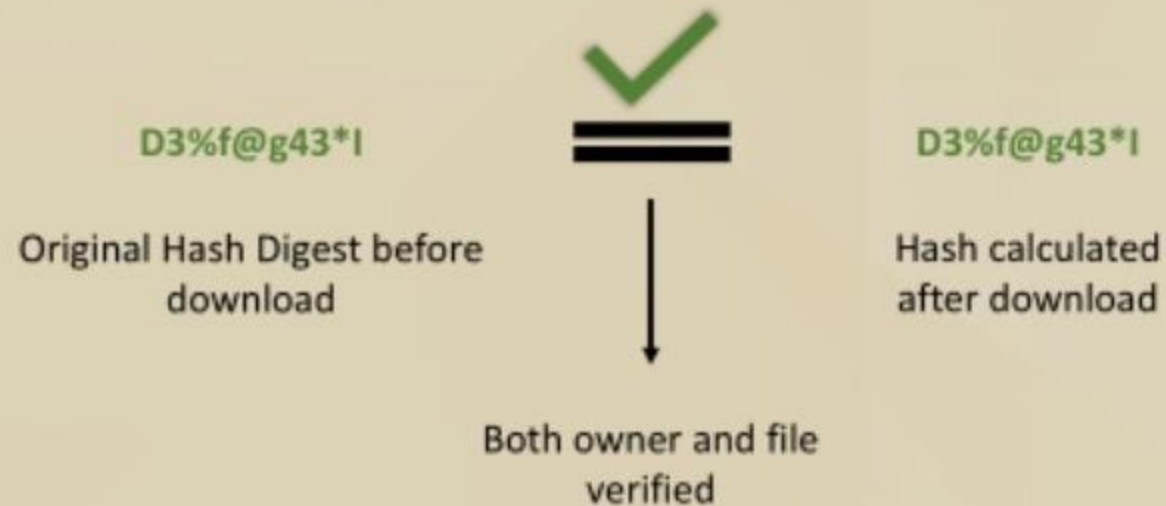
D3%f@g43*I

Hash Stored on the
Servers

Login is Denied

Real-World Implementation

Hashing can also be used for **integrity checks** to ensure the data isn't corrupted. The hash value/digest will always be the same for similar input.



Hash Functions



Hash Functions

- **Mathematical operations** to be carried out on two blocks of data.
- Both blocks are created by **dividing** the initial input into equal parts.
- **Irreversible** by design.
- Can be carried out multiple times, but the **final digest** must be consistent for the same input.

<u>Hash Algorithm</u>		<u>Digest Size</u>
MD5	→	128 bits
SHA-256	→	256 bits



Hashing Guidelines



Hashing Guidelines

Hash function must be fast, but not instantaneous



- Should be able to hash in-mass with a reasonable limit to **prevent exploitation**.
- Ultra quick algorithms can be tested rigorously for **brute force** attacks.
- With enough brute force attacks, not just the hash, **entire algorithm** can be cracked.

Hashing Guidelines



**Hash digest must be
dependent on each bit**

Hashing Guidelines

simpleart

Hash digest must be dependent on each bit



- If a single character changes, a **substantial portion** of the digest must change.
- Helpful in creating as many **unique hashes** as possible.
- Hash digest for the plaintext '**Cryptography**' will be **completely different** than when the plaintext is '**Cryptograph**'.

Hashing Guidelines



Prevent Hash Collision

Hashing Guidelines

Prevent Hash Collision



- Collision occurs when there are two exactly **same hash values/digests**.
- Since there is only one hash function for each server, **same passwords** have **same digests** after hashing.
- **Salting** can help prevent collisions, as we will learn later in this lesson.

Hashing Guidelines

Most Common Passwords of 2020

1. 123456
2. 123456789
3. picture1
4. password
5. 12345678
6. 11111111
7. 123123
8. 12345
9. 1234567890
10. Senha
11. 1234567
12. qwerty

Ways to Strengthen Hashes



Salting

- Salting is the process of **adding a random keyword** to the end of the input.
- The random keyword added is called the **salt/salt value**.
- The salt is **unique** for each user in the database and is helpful to battle hash collision.

Input into Hash Function	MD5 Hash Value/Digest
Cryptography	d2fc0657a64a3291826136c7712abbe7
Cryptographyabc123	c56db83ab5482b4e94536f4a29b21de0
Cryptographyxyz456	783b10b483435e05f3f2705bdd5a825c

Peppering

- Peppering is the process of adding the **same random value** at the end of a plaintext.
- Since it doesn't change per user, the random value **need not be stored** on server.
- In the case of a data breach, pepper value is **safe from further exploitation**.

Input into Hash Function	MD5 Hash Value/Digest
CryptographyRan123	4cac3f25fffad414e834ee8208f65116
MyPasswordRan123	470dd61e2acce64486e784f3a288d82f
Qwerty101Ran123	a8792a61ee831c39548ec1a2e1ba3d68